A2 Report

Team members: Ellen Chan (101303431), Joanna Joy (101303246)

1.

(d)

Table 1: Training accuracies with different hyper-parameters

λ	SqHinge			Hinge		
	Model1	Model2	Model3	Model1	Model2	Model3
0.001	0.9999	0.5734	0.8713	0.0119	0.605	0.1495
0.01	0.9988	0.5841	0.8712	0.0158	0.61	0.1503
0.1	0.9973	0.5708	0.875	0.0132	0.5976	0.1456
1.0	0.9906	0.576	0.8691	0.032	0.5988	0.1422

Table 2: Test accuracies with different hyper-parameters

λ	SqHinge			Hinge		
	Model1	Model2	Model3	Model1	Model2	Model3
0.001	0.98887	0.5538	0.88008	0.0115	0.60642	0.14127
0.01	0.98995	0.55182	0.87984	0.01247	0.60173	0.14153
0.1	0.98755	0.53509	0.88006	0.015	0.59401	0.13838
1.0	0.98225	0.54853	0.87897	0.02844	0.598	0.1327

(e) From both the tables above, we can see that both the test and training accuracies are lower with hinge, in comparison to squared hinge. This makes sense, because when calculating squared hinge, all the values are squared. We can also see that model one consistently has the smallest accuracy for hinge, but the opposite for squared hinge.

In addition to this, the larger the lambda, the simpler the model becomes, resulting in subpar squared hinge values, and the highest hinge values. Squared hinge seems to need a more complex model to have good accuracy.

Overall, we also see that for squared hinge model 1 with lambda as 0.01 yields the highest accuracy. As for hinge, while none are particularly high, model 2 with lambda as 0.001 yields the best results.

2.

(d)
Table 3: Training accuracies with different kernels

Kernel	SqHinge			Hinge		
	Model1	Model2	Model3	Model1	Model2	Model3
Linear	1.0	0.582	0.868	0.999	0.618	0.868
Poly(d=2)	1.0	1.0	0.869	0.998	1.0	0.874
Poly(d=3)	1.0	1.0	1.0	1.0	1.0	1.0
Gauss(σ=1)	1.0	1.0	1.0	0.999	1.0	1.0
Gauss(σ=0.5)	1.0	1.0	1.0	1.0	1.0	1.0

Table 4: Test accuracies with different kernels

Kernel	SqHinge			Hinge		
	Model1	Model2	Model3	Model1	Model2	Model3
Linear	0.9919	0.5451	0.8796	0.9916	0.592	0.878
Poly(d=2)	0.9826	0.9562	0.8772	0.981	0.956	0.8787
Poly(d=3)	0.9697	0.9626	0.9897	0.9698	0.9624	0.9894
Gauss(σ=1)	0.9613	0.9696	0.9964	0.961	0.9693	0.9962
Gauss(σ=0.5)	0.9654	0.9519	0.9977	0.9653	0.9516	0.9975

(e)
Training and test accuracies are generally similar, though training accuracy is often slightly higher because the models are fit to the training data. More flexible kernels may fit those examples especially well, which can lead to some overfitting.

Model 1 is linearly separable, which explains why the linear kernel performed very well. Model 2 has a nonlinear, circular-like decision boundary, which a linear kernel cannot represent, thus explaining its poor accuracy. Model 3 has two crescent-shaped classes, which

was not captured well by the linear or degree-2 polynomial kernel. The degree-3 and Gaussian kernels can represent more complex boundaries, thus they perform better.

The hinge and squared-hinged loss results were quite similar, suggesting the kernel choice and data shape have a larger effect on accuracy than the type of loss used.

3.

(c)
Table 5: Average validation accuracy

		Kernel				
		Linear	Poly(d=2)	Poly(d=3)	Gauss(σ=1)	Gauss(σ=0.5)
λ	0.001	0.963	0.978	0.971	0.505	0.479
	0.01	0.963	0.979	0.972	0.505	0.478
	0.1	0.963	0.979	0.972	0.505	0.478
	1.0	0.963	0.979	0.971	0.499	0.478

Best parameters: λ=1.0, kernel=polynomial degree 2, accuracy=0.979

I chose the configuration with the highest average validation accuracy. The degree-2 polynomial and Gaussian kernels tie at 0.979 for all three λ values. Both outperform the linear kernel at 0.963. So these results suggest that the nonlinear kernels are a better fit for this classification task, while λ had no measurable effect on accuracy among the values tested.

I chose λ = 1.0 with the degree-2 polynomial kernel as a tie-break since it achieves the top accuracy and uses the strongest regularization among the tested λ values.